



Agentic Coding with Antigravity CLI

Google's AI Agent in Your Terminal

Press Space for next page →

Contact Info

Ken Kousen Kousen IT, Inc.

- ken.kousen@kousenit.com
- <http://www.kousenit.com>
- <http://kousenit.org> (blog)
- Social Media:
 - [@kenkousen](https://twitter.com/kenkousen) (twitter)
 - [@kenkousen@foojay.social](https://mastodon.social/@kenkousen) (mastodon)
 - [@kousenit.com](https://kousenit.com) (bluesky)
- *Tales from the jar side* (free newsletter)
 - <https://kenkousen.substack.com>
 - <https://youtube.com/@talesfromthejarside>

Course Overview

- **Duration:** 5 hours of hands-on learning
- **Format:** Instructor-led with multiple labs
- **Hands-on Labs:** Real codebases in Python, JavaScript, Java
- **Prerequisites:** Command-line experience, development background

Topics Covered

- **Foundation:** Installation, CLI basics, authentication
- **Core Skills:** File operations, shell integration, context management
- **Customization:** AGENTS.md, skills, settings.json
- **Safety:** Execution modes, permission rules, sandbox, hooks
- **Advanced:** MCP integration, skills/plugins, session management

What's New in Antigravity CLI 1.1.x

- **Execution modes:** `Shift+Tab` cycles `default` → `accept-edits` → `plan` ; `--mode` flag (1.1.0)
- **Structured headless output:** `--output-format json|stream-json` , `--json-schema` (1.1.8)
- **Slash commands in** `-p` : skills expand; read-only commands answer without spending quota (1.1.9–1.1.12)
- **Model slugs +** `--effort` : pin `gemini-3.1-pro-high` , tune reasoning with `/effort` (1.1.5)
- **Custom agents in Markdown:** `agent.md` with YAML frontmatter; `--agent` , `agy agents` (1.1.1/1.1.6)
- `/codesearch` (1.1.3), **Vim editor mode** (1.1.11), `/fork` , `/btw` , `/rewind`
- **Direct Gemini API:** `GEMINI_API_KEY` + `modelProvider: "gemini"` — no sign-in (1.1.13)
- **Latest stable track:** Antigravity CLI `1.1.13` — see `agy changelog`

What is Antigravity CLI?

- Google's Go-based AI coding agent for the terminal (binary: `agy`)
- **Multi-model:** Gemini 3.x, Claude Sonnet/Opus 4.6, GPT-OSS — switch with `/model`
- Built-in tools: web search, file ops, shell, web fetch
- Model Context Protocol (MCP) support via `mcp_config.json`
- Same `~/.gemini` config family as the Antigravity 2.0 desktop app and IDE

Key Differentiators

- **Model choice:** mix Gemini, Claude, and GPT-OSS in one tool
- **Google Search Grounding:** real-time web access
- **Free tier + G1 credits:** keep working when quota runs out
- **Desktop continuity:** same account, quota, and `~/.gemini` config as Antigravity 2.0
- **MCP Native:** built-in Model Context Protocol support

Models Available

- **Gemini 3.7 / 3.6 / 3.5 Flash:** fast Gemini models, each in Low/Medium/High effort
- **Gemini 3.1 Pro:** high-capability Gemini model for complex coding (Low/High)
- **Claude Sonnet 4.6 / Opus 4.6:** Anthropic models (thinking)
- **GPT-OSS 120B:** open-weights option
- **List models:** `agy models` · **Switch:** `/model` or `--model <slug>`
- **Reasoning effort:** `--effort low|medium|high` or `/effort mid-session`

Quota & Credits

- **Free tier:** sign in with a Google account to get started
- **G1 credits:** kick in automatically when standard quota runs out
- **In-CLI panels:** `/credits` , `/usage` , `/quota` for real-time status
- Manage models and preferences via `/settings`

📖 **Docs:** antigravity.google/docs

Installation

- **macOS / Linux:** `curl -fsSL https://antigravity.google/cli/install.sh | bash`
- **Windows (PowerShell):** `irm https://antigravity.google/cli/install.ps1 | iex`
- Installs the `agy` binary to `~/.local/bin/`
- Verify: `agy --version` · Update: `agy update` · What changed: `agy changelog`

```
1  # Install (macOS / Linux)
2  curl -fsSL https://antigravity.google/cli/install.sh | bash
3
4  # Verify installation
5  agy --version
```

Authentication

- **Google Sign-In** (default): launches automatically on first run
- **Remote / SSH**: shows an authorization URL with a one-time code
- **API key (alternative)**: `export ANTIGRAVITY_API_KEY="your-key"`
- **Direct Gemini API** (1.1.13): `GEMINI_API_KEY + "modelProvider": "gemini"` in settings — no sign-in
- Enterprise: Business sign-in, Workforce Identity Federation, ADC (1.1.10)
- Credentials persist via OAuth in `~/.gemini/`

```
1  # First run: sign in with your Google account
2  agy
3
4  # Alternative: API key for scripts / headless use
5  export ANTIGRAVITY_API_KEY="your-api-key"
6  agy -p "Summarize this repo"
7
8  # Alternative: talk to the Gemini API directly (no Google sign-in)
9  export GEMINI_API_KEY="your-gemini-key"      # plus "modelProvider": "gemini" in settings.json
```

Basic Usage Modes

- **Interactive REPL:** `agy` - Start a conversation
- **One-shot / print:** `agy -p "prompt"` - Single response
- **Piped input:** `echo "task" | agy -p`
- **Interactive with initial prompt:** `agy -i "initial context"`

```
1  # Interactive mode
2  agy
3
4  # One-shot (print) mode
5  agy -p "Explain what this codebase does"
6
7  # Start interactively with an initial prompt
8  agy -i "You are a Python expert"
```

Core Features

Essential Capabilities

Master the fundamentals



File References with @

- Reference files directly: `@./src/main.js`
- Reference directories: `@./src/` (recursive)
- Reference images: `@./screenshot.png`
- Multiple references in one prompt

```
1  # Reference a specific file
2  agy -p "Explain @./src/app.py"
3
4  # Reference multiple files
5  agy -p "Compare @./old.js and @./new.js"
6
7  # Reference a directory
8  agy -p "Analyze the architecture in @./src/"
```

Shell Integration with !

- Execute shell commands: `!git status`
- Toggle persistent shell mode: `!`
- The agent can observe and analyze output
- Combine with AI analysis

```
1  # In interactive mode:
2  > !npm test
3  # The agent sees the test output
4
5  > !git diff
6  # Ask the agent to analyze the changes
7
8  # Toggle persistent shell mode
9  > !
```

Search the Workspace: `/codesearch`

- `/codesearch <query>` (aliases `/cs` , `/search`) — interactive regex search across the workspace
- `-F / --literal` for exact matching; `f: / file:` globs to include or exclude paths
- Falls back to a local search if `ripgrep` can't run (e.g. blocked by endpoint security)
- Cheaper than asking the agent to "find where X is used" — and you see the results yourself

```
1 > /cs def get_weather
2 > /cs -F "TODO:" file:*.py
```

Slash Commands: Navigation

- `/help` - Show available commands and shortcuts
- `/context` - View loaded context and token usage
- `/model` · `/effort` - Switch model or reasoning effort mid-session
- `/settings` - Open settings and preferences
- `/codesearch <query> ( /cs )` - Regex search across the workspace

Slash Commands: Workflow

- `/agents` · `/tasks` - Monitor subagents and background tasks
- `/btw <question>` - Ask a side question without interrupting the current task
- `/permissions` - Add/edit/remove tool permission rules
- `/mcp` - View and reload MCP servers
- `/usage` - Session, quota, and rate-limit status

Slash Commands: Session Control

- `/resume` - Open the conversation browser
- `/fork` - Clone the conversation into a parallel session
- `/rewind` - Roll back to an earlier message
- `/credits` · `/quota` - Credit balance and quota panels

Slash Commands in Action (Context)

```
1  # Show what context is loaded and token usage
2  /context
3
4  # Switch the active model
5  /model
6
7  # Manage tool permission rules
8  /permissions
9
10 # Open settings and preferences
11 /settings
```

Slash Commands in Action (Sessions)

```
1  # Open the conversation browser
2  /resume
3
4  # Branch this conversation into a parallel session
5  /fork
6
7  # Ask a quick side question without derailing the current task
8  /btw what does the retry decorator in @./src/utils.py do?
```

Keyboard Shortcuts: Editing

- `Ctrl+L` - Clear screen
- `Ctrl+V` - Paste text/images
- `Ctrl+R` - Open the Artifact Review panel

Keyboard Shortcuts: Control

- `Esc` - Interrupt the active agent stream
- `Ctrl+C` - Cancel current operation
- `Ctrl+D` `Ctrl+D` - Exit Antigravity CLI (press twice)
- `Ctrl+G` - Edit a long prompt in `$EDITOR`

Vim Editor Mode

- `/settings` → **Editor Mode** → `vim` (or `"editorMode": "vim"` in `settings.json`)
- Normal / Insert / Visual modes, a mode badge in the status line
- Motions, operators, text objects: `w` `b` `e` `0` `$` `gg` `G` `d` `c` `y` `iw` `ap` ...
- **Insert First** option: open each prompt in Insert mode so bare `Enter` submits
- Remap anything under `vim.*` in `~/.gemini/antigravity-cli/keybindings.json`

```
1  # Persist it
2  { "editorMode": "vim", "vimInsertFirst": true }
```

Built-in Tools

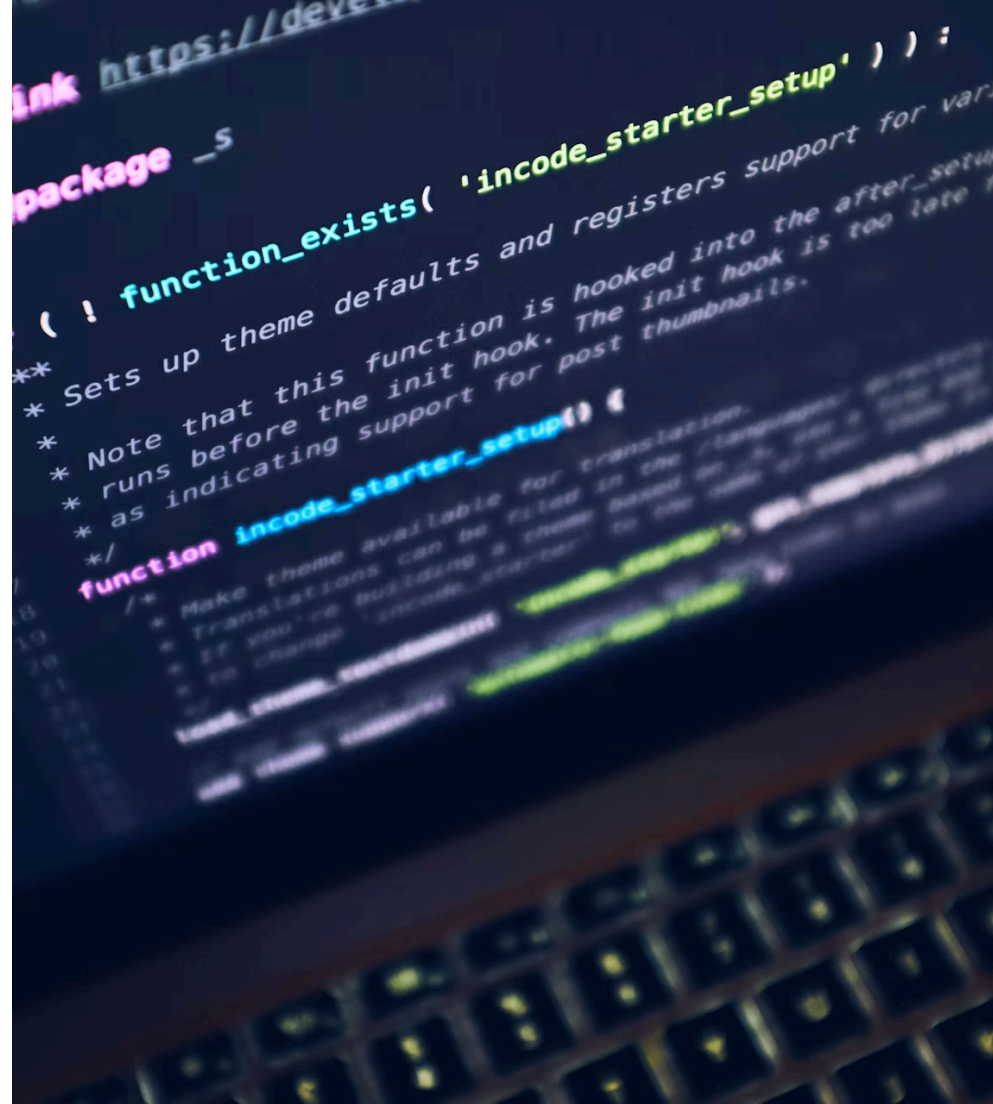
- **File System:** `read_file()`, `write_file()`, `replace()`, `glob()`
- **Shell:** Execute terminal commands
- **Web:** `google_web_search()`, `web_fetch()`
- **Memory:** `save_memory()` for cross-session recall

```
1  # The agent automatically uses appropriate tools
2  "Search the web for React 19 new features"
3  # Uses google_web_search()
4
5  "Read all Python files in src/"
6  # Uses glob() and read_file()
7
8  "Update the README with the changes we made"
9  # Uses write_file()
```

Safety & Control

Work Safely

Permission rules and sandboxing



Tool Permissions

- **Default** (`request-review`): prompt for approval on each tool call
- `/permissions` : add, edit, or remove allow/deny rules in-CLI
- `toolPermission` setting: `request-review` · `proceed-in-sandbox` · `strict` · `always-proceed`
- `--dangerously-skip-permissions` : auto-approve everything (use with care)

```
1  # Default - interactive, prompts per tool call
2  agy
3
4  # Auto-approve all tool calls (use with care)
5  agy --dangerously-skip-permissions
6
7  # Manage allow/deny rules from inside a session
8  > /permissions
```

Execution Modes

- `default` : pauses for an interactive diff review before writing files (`f` = full diff)
- `accept-edits` : auto-approves file edits and creations
- `plan` : prepends `/plan` — analyze and outline before writing code
- **Cycle** with `Shift+Tab` ; the current mode shows in the status line
- `/planning` and `/fast` were removed in 1.1.0

```
1  # Pick a mode at launch
2  agy --mode accept-edits
3  agy --mode plan
4
5  # Persist a default in ~/.gemini/antigravity-cli/settings.json
6  { "agentMode": "accept-edits" }
```

Artifact Review

- Review proposed changes before they are applied
- Open the **Artifact Review panel** with `Ctrl+R`
- Inspect diffs with `/diff` (supports commit-hash selection)
- Works even while answering pending tool-permission prompts
- Great safety habit when exploring an unfamiliar codebase

```
1  # Inspect changes from inside a session
2  > /diff
3
4  # Toggle the Artifact Review panel
5  Ctrl+R
```

Sandbox Mode

- Isolate file operations away from your host
- Multiple backends depending on your OS
- Prevents accidental system changes
- Perfect for exploring unfamiliar code

```
1  # Run in sandbox mode (terminal restrictions enabled)
2  agy --sandbox
3
4  # Combine with a one-shot prompt
5  agy --sandbox -p "Refactor this entire codebase"
```

Sandbox & Permission Modes

- `--sandbox` : run with terminal restrictions enabled
- `proceed-in-sandbox` permission mode: auto-approve commands that stay inside the sandbox, prompt only when one tries to break out
- Sandbox isolation is enforced in headless print mode too (`-p`)
- Pair with `/permissions` rules for durable guardrails

```
1  # Interactive, sandboxed
2  agy --sandbox
3
4  # Non-interactive, sandbox still enforced
5  agy --sandbox -p "Audit @./src for risky calls"
```

 antigravity.google/docs

Undo & Recovery

- **Before the write:** the default mode pauses on a diff — reject what you don't want
- `/rewind` : roll the conversation back to an earlier message
- `/diff` : see what the agent changed in this session
- `/fork` : branch before a risky experiment; `/resume` / `agy -c` to come back
- **Your repo is the real safety net:** commit before big asks, `git checkout -- .` to undo

```
1  # Roll the conversation back, then try a different approach
2  > /rewind
3
4  # Inspect what changed before you commit
5  > /diff
6  git diff
```



Context Management

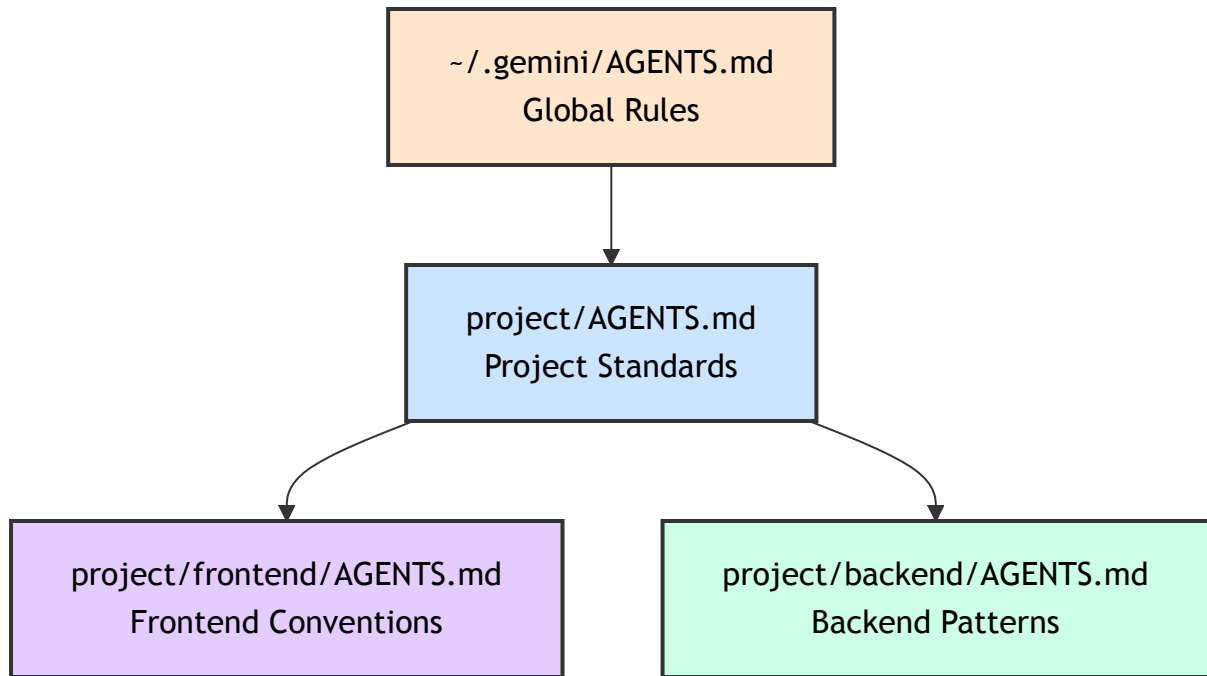
AGENTS.md Files

Project memory and instructions

What is AGENTS.md?

- **Project memory** loaded automatically
- **Coding standards** and conventions
- **Architecture context** for the AI
- **Persistent instructions** across sessions
- Antigravity also recognizes `GEMINI.md` and `CLAUDE.md`

Hierarchical Loading



More specific files override general ones

Example AGENTS.md

```
1  # Project: Weather API
2  ## Tech Stack
3  - Backend: Python Flask
4  - Database: PostgreSQL
5  - Testing: pytest
6  ## Coding Standards
7  - Use type hints for all functions
8  - Follow PEP 8 style guide
9  - Write docstrings for public APIs
10 ## Current Focus
11 Implementing caching layer for API responses
```

Managing Context

- Edit `AGENTS.md` directly to update project memory
- `/context` - view combined context and token usage
- Add `includeDirectories` in settings to load shared context
- Restart or reload the session to pick up changes

```
1  # Create or edit project memory
2  $EDITOR AGENTS.md
3
4  # See what context is loaded
5  > /context
```

Modular Imports

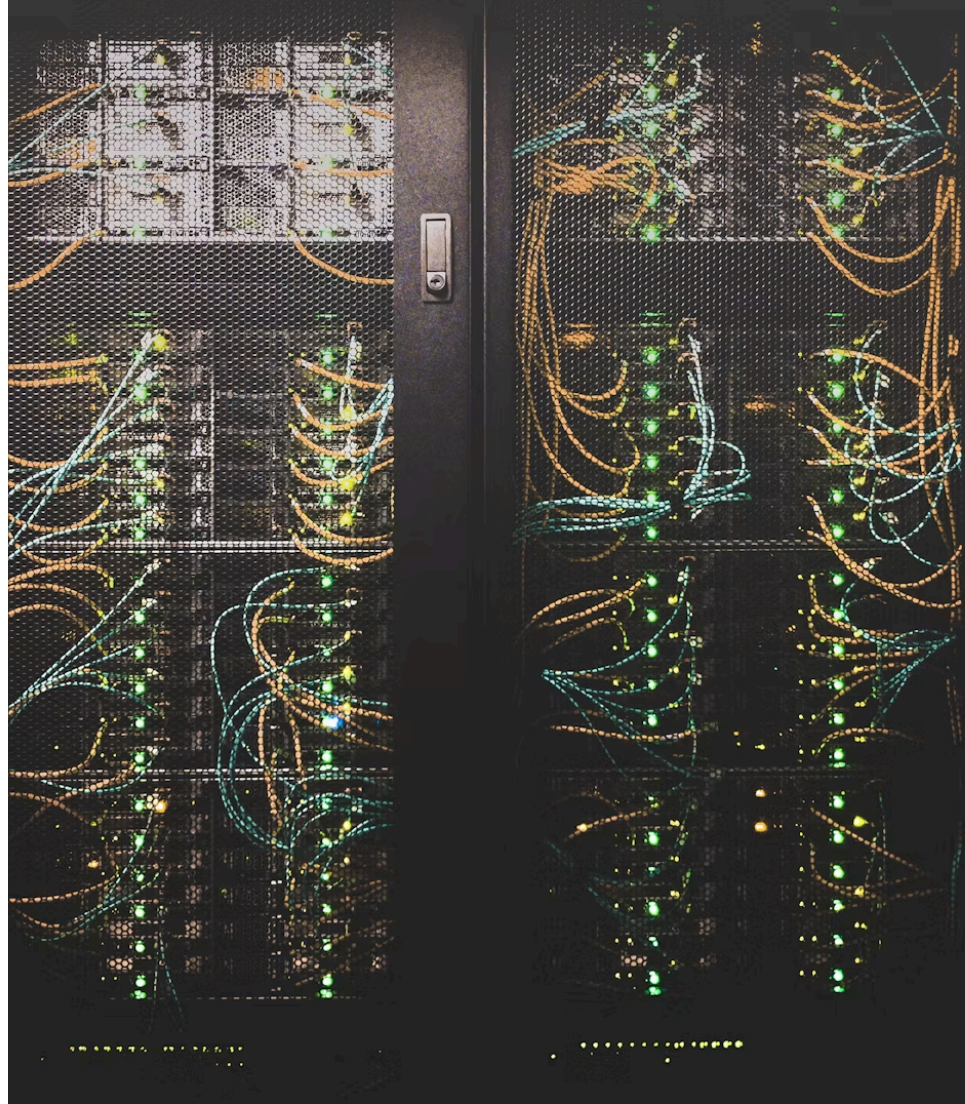
- Import other files with `@file.md` syntax
- Break large context into components
- Supports relative and absolute paths

```
1  # AGENTS.md
2
3  ## Project Overview
4  @./docs/architecture.md
5
6  ## Coding Standards
7  @./docs/style-guide.md
8
9  ## API Documentation
10 @./docs/api-reference.md
```

Configuration

Customize Your Setup

settings.json and environment



Configuration Layers

1. **Default values** - Built-in defaults
2. **User settings** - `~/.gemini/antigravity-cli/settings.json` (edit via `/settings`)
3. **Workspace files** - `.agents/mcp_config.json` , `.agents/agents/` , `AGENTS.md`
4. **Environment variables** - e.g. `GEMINI_API_KEY` , `AGY_CLI_CMD_OUTPUT_PERCENTAGE`
5. **Command-line arguments** - `--mode` , `--model` , `--effort` , ... highest priority

settings.json Options

`~/.gemini/antigravity-cli/settings.json` — a flat key/value file:

```
1  {
2    "agentMode": "accept-edits",
3    "toolPermission": "request-review",
4    "artifactReviewPolicy": "asks-for-review",
5    "enableTerminalSandbox": false,
6    "editor": "auto",
7    "editorMode": "vim"
8  }
```

settings.json Options (UI + Permissions)

```
1  {  
2    "showTips": true,  
3    "notifications": true,  
4    "colorScheme": "terminal",  
5    "permissions": {  
6      "allow": ["command(git status)", "command(npm test)"]  
7    },  
8    "trustedWorkspaces": ["/path/to/project"]  
9  }
```

Full list: `/settings` in-session, or `agy -p "/settings"` to dump current values.

Status Line Customization

- The bar under the prompt shows model, mode, and quota by default
- `/statusline` opens the customization overlay
- Or point `statusLine` at your own script — it receives session JSON on stdin

```
1  {  
2    "statusLine": {  
3      "type": "command",  
4      "command": "~/.local/bin/agy-statusline.sh",  
5      "enabled": true  
6    }  
7  }
```

A working Python example lives in `exercises/python/weather-app/statusline.py` .

Tool Permissions & Rules

- `/permissions` : add, edit, or remove allow/deny rules in-CLI
- Rules merge across three layers: project, user, and CLI settings
- Shared with the Antigravity desktop app's permission settings
- Keep high-risk tools constrained for teams

```
1  # Manage permission rules interactively
2  > /permissions
3
4  # Auto-approve everything (use with care)
5  agy --dangerously-skip-permissions
```

 antigravity.google/docs

File Filtering

- **respectGitIgnore:** Honor .gitignore patterns
- **enableRecursiveFileSearch:** Recursive completion
- Exclude rules and allowlists live in `rules.json`

```
1  {  
2    "context": {  
3      "fileFiltering": {  
4        "respectGitIgnore": true,  
5        "enableRecursiveFileSearch": true  
6      }  
7    }  
8  }
```

Environment Variables

Variable	Purpose
ANTIGRAVITY_API_KEY	API-key authentication (alternative to sign-in)
AGY_CLI_DISABLE_LATEX	Turn off LaTeX math rendering
AGY_CLI_HIDE_ACCOUNT_INFO	Hide email and plan tier from the header
\$EDITOR	External editor for prompts and files
HTTP_PROXY	Network proxy

Custom Context Filenames

- `AGENTS.md` is the documented default context file
- `GEMINI.md` and `CLAUDE.md` are also recognized
- Global rules: `~/.gemini/AGENTS.md` ; per-directory files nest under the project
- Add extra directories to the workspace with `--add-dir` or `/add-dir`

```
1  # Bring shared context from another directory into the workspace
2  agy --add-dir ~/shared-context
3
4  # ...or mid-session
5  > /add-dir ~/shared-context
```

Advanced Features

Power User Tools

MCP, Plugins, Sessions



Model Context Protocol (MCP)

- Standard protocol for AI-to-system connections
- Connect to external tools and services
- Supports local commands, HTTP, and SSE
- OAuth 2.0 for remote authentication

MCP Configuration: Firecrawl

```
1  {
2    "mcpServers": {
3      "firecrawl": {
4        "command": "npx",
5        "args": ["-y", "firecrawl-mcp"],
6        "env": {
7          "FIRECRAWL_API_KEY": "${FIRECRAWL_API_KEY}"
8        }
9      }
10   }
11 }
```

MCP Configuration: Database

```
1  {  
2    "mcpServers": {  
3      "postgres": {  
4        "command": "npx",  
5        "args": ["@modelcontextprotocol/server-postgres"],  
6        "env": {  
7          "CONNECTION_STRING": "${DATABASE_URL}"  
8        }  
9      }  
10   }  
11 }
```

Managing MCP Servers

Edit the config file directly, then reload from `/mcp` :

```
1  # Global config
2  $EDITOR ~/.gemini/config/mcp_config.json
3
4  # Per-project config (checked into the repo)
5  $EDITOR .agents/mcp_config.json
```

```
1  {
2    "mcpServers": {
3      "github": {
4        "command": "npx",
5        "args": ["-y", "@modelcontextprotocol/server-github"]
6      }
7    }
8  }
```

Servers initialize in parallel on startup.

MCP: Remote Servers

Use `serverUrl` for HTTP/SSE remote MCP servers (`url` / `httpUrl` are **not** supported):

```
1  {
2    "mcpServers": {
3      "remote-tools": {
4        "serverUrl": "https://mcp.example.com/sse",
5        "headers": { "Authorization": "Bearer ${MY_TOKEN}" }
6      }
7    }
8  }
```

Per-project servers go in `.agents/mcp_config.json` . Reload with `/mcp` .

MCP Server Options

- **command**: Shell command to start server
- **args**: Command arguments
- **env**: Environment variables
- **cwd**: Working directory

MCP Server Options (Advanced)

- `disabled` : keep a server configured but switched off
- `disabledTools` : withhold specific tools from the agent
- `headers` / `oauth` / `authProviderType: "google_credentials"` : remote auth
- `env` , `cwd` : environment and working directory for stdio servers

Popular MCP Servers

- **Firecrawl** - Web scraping, search, content extraction
- **PostgreSQL** - Database queries and schema
- **Filesystem** - Extended file operations
- **Slack** - Team communication
- **Playwright** - Browser automation

📖 **Registry:** modelcontextprotocol.io/registry

Plugins from Marketplaces

- Install from a marketplace with `plugin@marketplace` syntax
- Plugins bundle **skills** and **subagents**, discovered automatically
- Import existing Gemini or Claude plugins with `agy plugin import`
- Plugins install to the shared `~/.gemini/config/` directory

```
1  # Install, then link a marketplace
2  agy plugin install my-plugin@my-marketplace
3  agy plugin link my-marketplace ./target
```

 antigravity.google/docs

Subagents

- Ask the agent to **delegate**: it dispatches a subagent via `invoke_subagent`
- Define your own in Markdown: `.agents/agents/<name>.md` (or `~/.gemini/config/agents/`)
- Frontmatter: `subagent: true` , `mainAgent` , `model: flash|pro|inherit` , `hidden` , `inheritMcp`
- Monitor with `/agents` ; background shell tasks with `/tasks` ; ask a side question with `/btw`
- Launch straight into a custom agent: `agy --agent <name>` · list with `agy agents`

```
1  ---
2  subagent: true
3  model: flash
4  ---
5  # Test writer
6  You write focused unit tests. Never modify source files.
```

Plugin System

- Extend Antigravity CLI with **plugins** (skills + subagents)
- Managed with the `agy plugin` subcommand
- Import from Gemini or Claude: `agy plugin import gemini`
- Installed plugins are scanned for skills and agents automatically

```
1  # List installed plugins
2  agy plugin list
3
4  # Install / enable / disable
5  agy plugin install my-plugin@marketplace
6  agy plugin enable my-plugin
7  agy plugin disable my-plugin
```

Skills

- Package reusable expert workflows as **skills**: a folder with a `SKILL.md`
- Frontmatter `name` + `description` — the agent reads the description to decide when to use it
- Live in `.agents/skills/` (project, checked in) or `~/.gemini/config/skills/` (global), or inside plugins
- Surface as `/name` slash commands via autocomplete; `/skills` lists them
- This repo ships four in `skills/`: `/review`, `/test-gen`, `/docs`, `/refactor`

```
1  ---
2  name: review
3  description: Perform a code review. Use when the user runs /review on a file.
4  ---
5  # /review
6  Review the referenced code for security, performance, and test gaps...
```

Lifecycle Hooks

- Shell commands that run at fixed points in the agent loop — `hooks.json` in `~/.gemini/config/` or inside a plugin
- `PreToolUse` — gate or rewrite a tool call: return `allow` / `deny` / `ask` (matcher on tool name, e.g. `run_command`)
- `PostToolUse` — run a linter or tests after a tool finishes
- `PreInvocation` / `PostInvocation` — inject context before the model runs; force-continue after
- `Stop` — notify, or refuse to stop until goals are met
- JSON in on stdin, JSON out on stdout; `/hooks` shows what's loaded

Hooks: Examples

```
1  {
2    "no-force-push": {
3      "PreToolUse": [{
4        "matcher": "run_command",
5        "hooks": [{ "command": "./scripts/block-force-push.sh" }]
6      }]
7    },
8    "notify-done": {
9      "Stop": [{ "command": "osascript -e 'display notification \"agy finished\"; echo '{}'" }]
10   }
11 }
```

```
1  # block-force-push.sh - stdin has {"toolCall":{"name":"run_command","args":{"CommandLine":"..."}}}
2  if grep -q 'push.*--force' <<< "${cat}"; then
3    echo '{"decision":"deny","reason":"No force pushes from the agent"}'
4  else
5    echo '{"decision":"allow"}'
6  fi
```

Example `config-examples/hooks.json` in this repo. ⚠️ As of 1.1.13 the CLI loads hooks from `~/.gemini/config/` and plugins — a workspace `.agents/hooks.json` is documented but not picked up yet (verified).

Session Management

- **Continue:** pick up the most recent conversation
- **Resume by ID:** jump to a specific past conversation
- **Browse:** open the conversation picker with `/resume`
- Conversations are stored in SQLite (`.db`)

```
1  # Continue the most recent conversation
2  agy -c
3
4  # Resume a specific conversation by ID
5  agy --conversation <id>
6
7  # Browse conversations from inside a session
8  > /resume
```

Non-Interactive (Print) Mode

- `-p / --print` : run a single prompt and print the response
- `--output-format` : `text` (default) · `json` (one envelope) · `stream-json` (NDJSON events)
- `--json-schema` : constrain the answer to a schema (inline or file path)
- `--print-timeout` : bound how long print mode waits (default 5m)
- Skills expand in `-p` (`agy -p "/my-skilll ..."`); read-only commands like `/settings` , `/quota` answer without spending quota

```
1  # One-shot print mode, piped input
2  cat error.log | agy -p "Diagnose this stack trace"
3
4  # Machine-readable: {"status":"SUCCESS","response":"...","usage":{"..."}}
5  agy -p "Summarize @./README.md" --output-format json
6
7  # Enforce a schema; result lands in "structured_output"
8  agy -p "Extract version from @./package.json" --output-format json \
9      --json-schema '{"type":"object","properties":{"version":{"type":"string"}}}'
```

Desktop Integration

- **Antigravity 2.0 desktop app** and IDE live alongside the CLI under `~/.gemini`
- **Shared MCP config:** `~/.gemini/config/mcp_config.json`
- **Shared permissions:** rules carry across CLI and desktop
- **External editor:** set `$EDITOR` for prompt and file editing

```
1  # Open a file in your external editor from inside a session
2  > /open src/app.py
3
4  # Long prompt? ctrl+g opens $EDITOR for the prompt box
```



Practical Applications

Real-World Workflows

Common use cases

Code Exploration

- Understand unfamiliar codebases
- Trace dependencies and data flow
- Find patterns and conventions
- Generate architecture documentation

```
1 agy -p "Analyze the architecture of @./src/ and explain  
2 how the components interact"  
3  
4 agy -p "Trace the flow from the API endpoint to the database  
5 in @./src/controllers/ and @./src/services/"
```

Test Generation

- Generate unit tests for existing code
- Identify edge cases automatically
- Create integration test scaffolding
- Mock setup and fixtures

```
1 agy -p "Create comprehensive unit tests for @./src/utils.py  
2 with pytest, including edge cases"  
3  
4 agy -p "Generate integration tests for @./src/api/users.py  
5 with proper mocking"
```

Documentation Generation

- README files for projects
- API documentation
- Architecture diagrams (Mermaid)
- Code comments and docstrings

```
1 agy -p "Generate a comprehensive README.md for this project"
2
3 agy -p "Add detailed docstrings to all public functions
4 in @./src/services/"
5
6 agy -p "Create a Mermaid diagram showing the system architecture"
```

Refactoring & Modernization

- Upgrade legacy code patterns
- Apply modern language features
- Improve code organization
- Fix anti-patterns

```
1 agy -p "Refactor @./src/legacy.py to use modern Python 3.12  
2 features like type hints and match statements"  
3  
4 agy -p "Convert this callback-based code to async/await  
5 @./src/api.js"
```

Debugging Workflows

- Analyze error messages and stack traces
- Identify root causes
- Suggest fixes with context
- Test and verify solutions

```
1  agy -p "This test is failing with @./tests/output.log.  
2  Analyze the error and fix the issue in @./src/app.py"  
3  
4  agy -p "Debug why the API returns 500 errors.  
5  Check @./src/routes.py and @./src/middleware.py"
```

Git Workflows

- Generate commit messages
- Create pull request descriptions
- Analyze diffs and changes
- Resolve merge conflicts

```
1  # Analyze staged changes
2  !git diff --staged
3  "Generate a conventional commit message for these changes"
4
5  # Create PR description
6  "Create a pull request description summarizing the changes
7  from the last 5 commits"
```

CI/CD Integration

- Non-interactive `--print` mode for pipelines
- Capture stdout to a file for parsing
- Exit codes for success/failure
- Automated code reviews

```
1  # In CI/CD pipeline
2  agy -p "Review @./src/ for security issues" > review.txt
3
4  # Check exit code
5  if agy -p "Verify all tests pass"; then
6      echo "All checks passed"
7  fi
```

Optional Advanced Module

- Designed for teams moving from personal usage to production workflows
- Can be taught live or assigned as follow-up practice
- Focus areas: auth strategy, governance, skills/MCP, CI automation



Authentication Matrix (Team Use)

- **Google Sign-In:** best default for local interactive usage
- **Device-code flow:** shows a URL + code for remote/SSH machines
- `ANTIGRAVITY_API_KEY` : scripts and headless automation
- **G1 credits:** keep teams productive past standard quota

Auth Quick Checks

```
1  # Interactive route: sign in with Google on first run
2  agy
3
4  # API-key route (scripts / headless)
5  export ANTIGRAVITY_API_KEY="..."
6  agy -p "Summarize @./src"
7
8  # Hide account info from the header (e.g. for screen sharing)
9  export AGY_CLI_HIDE_ACCOUNT_INFO=1
10 agy
```

Governance: Permissions and Rules

- `--dangerously-skip-permissions` is a per-session escape hatch
- `/permissions` rules provide durable guardrails across users/projects
- `PreToolUse` hooks (`~/.gemini/config/hooks.json` , or shipped in a plugin) enforce policy the rules can't express
- Keep high-risk tools constrained in team settings
- Prefer explicit allow/deny patterns over ad-hoc approvals

Governance: Folder Trust

- Antigravity tracks trusted folders (`trustedWorkspaces` in `settings.json`)
- Folder trust impacts settings, skills, and context loading
- Treat untrusted repos with `--sandbox` and stricter rules
- Combine with `/permissions` for auditable guardrails

Skills + MCP at Scale

- Skills package repeatable expert workflows for teams
- MCP connects external systems (docs, data, browsers, APIs)
- Distribute skills and subagents together inside plugins
- Use `disabledTools` to narrow risky MCP exposure

Plugins + MCP Commands

```
1  # Plugin (skills + subagents) lifecycle
2  agy plugin list
3  agy plugin disable my-plugin
4  agy plugin enable my-plugin
5
6  # MCP servers: edit config, then restart to reload
7  $EDITOR ~/.gemini/config/mcp_config.json
```

Automation Patterns (Non-Interactive)

- Prefer `--print` for non-interactive runs
- Use exit codes for pipeline gates
- Keep prompts deterministic and repo-scoped
- Store logs/artifacts for auditability

CI Example: Gate + Report

```
1  # Human-readable report for the PR
2  agy -p "Review @./src/ for security issues" > review.txt
3
4  # Machine-readable gate: force a yes/no answer with a schema
5  verdict=$(agy -p "Does @./src/ contain any CRITICAL security issue?" \
6    --output-format json \
7    --json-schema '{"type":"object","properties":{"critical":{"type":"boolean"}}}' \
8    | jq -r '.structured_output.critical')
9
10 [ "$verdict" = "false" ] || { echo "Gate failed - see review.txt"; exit 1; }
```

Non-zero exit = the run itself failed (`status` $\neq$ `SUCCESS`); the schema gives you the *verdict*.

Best Practices

Professional Workflows

Tips for success



Effective Prompting

- **Be specific** about what you want to achieve
- **Provide context** about goals and constraints
- **Use file references** to ground the conversation
- **Iterate** for complex tasks
- **Include examples** when possible

Safety Guidelines

- **Start in default mode** - Get comfortable first
- **Enable checkpointing** - Safety net for mistakes
- **Review generated code** - Don't blindly accept
- **Use sandbox** for exploratory work
- **Commit often** - Git is your safety net

Project Setup

1. Create a comprehensive AGENTS.md
2. Set up project-specific settings.json
3. Configure relevant MCP servers
4. Create skills for common tasks
5. Establish team conventions

AGENTS.md Best Practices

- **Keep it focused** - Relevant project info only
- **Update regularly** - Reflect current state
- **Use imports** - Modularize large contexts
- **Include examples** - Show expected patterns
- **Document conventions** - Style guides, patterns

Team Collaboration

- Share AGENTS.md in version control
- Standardize settings.json across team
- Create shared skills in `.agents/skills/`
- Document AI-assisted workflows
- Review AI-generated code together

Common Pitfalls

- **Overly broad prompts** → Be specific
- **Missing context** → Use AGENTS.md
- **Skipping review** → Always verify output
- **Skipping permissions** → Don't reach for `--dangerously-skip-permissions` early
- **Ignoring Artifact Review** → Inspect diffs before applying

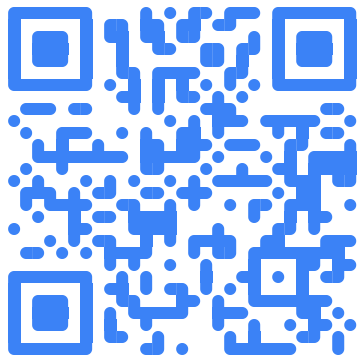
Troubleshooting

- **Authentication issues:** Re-run `agy` to sign in, or check `ANTIGRAVITY_API_KEY`
- **Rate limits:** Check `/quota` ; G1 credits cover overflow
- **Tool failures:** Verify `mcp_config.json` and server status
- **Context not loading:** Confirm `AGENTS.md` location, reload session
- **Logs:** Override the log path with `--log-file`

```
1  # Write logs to a known path for troubleshooting
2  agy --log-file ./agy.log
3
4  # Inspect loaded context and token usage
5  > /context
```

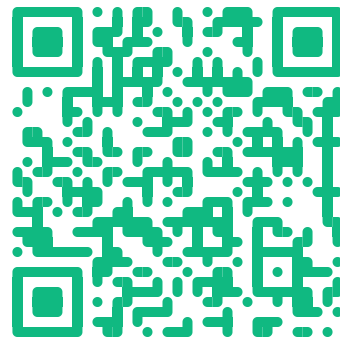
Quick Access

Antigravity CLI Docs



antigravity.google/docs

Course Repository



github.com/kousen/gemini-training

Important Links



Official Documentation

<https://antigravity.google/docs>



Source Repository

<https://github.com/google-antigravity/antigravity-cli>



MCP Server Registry

<https://modelcontextprotocol.io/registry>



Course Materials

<https://github.com/kousen/gemini-training>

Command Reference: Basic Usage

```
1  # Interactive mode
2  agy
3
4  # One-shot (print) mode
5  agy -p "prompt"
6
7  # Interactive with initial prompt
8  agy -i "context"
```

Command Reference: Safety

```
1  # Run in sandbox mode
2  agy --sandbox
3
4  # Auto-approve all tool calls (use with care)
5  agy --dangerously-skip-permissions
6
7  # Manage permission rules in-session
8  > /permissions
```

Command Reference: Sessions

```
1  # Continue the most recent conversation
2  agy -c
3
4  # Resume a conversation by ID
5  agy --conversation <id>
6
7  # Browse conversations in-session
8  > /resume
```

Command Reference: Output

```
1  # Print mode for scripting
2  agy -p "prompt"
3
4  # Bound how long print mode waits
5  agy --print-timeout 2m -p "prompt"
6
7  # Write logs to a file
8  agy --log-file ./agy.log
```

Thank You!

Questions?



Kenneth Kousen *Author, Speaker, Java & AI Expert*

kousenit.com | [@kenkousen](https://twitter.com/kenkousen)